Rozproszone bazy danych

Łącza bazodanowe

1. Z poziomu systemu operacyjnego wyświetl wartość zmiennej środowiskowej ORACLE_SID, zawierającej nazwę lokalnej bazy danych, domyślnej przy łączeniu się z bazą danych za pomocą `sqlplus` bez jawnego wskazania nazwy bazy danych.

   ```
   echo $ORACLE_SID
   ```

2. Podejrzyj zawartość pliku tnsnames.ora, zawierającego specyfikacje zdalnych baz danych (nazwy „usług" i odpowiadające im parametry połączenia). Plik ten znajduje się w domyślnej lokalizacji `$ORACLE_HOME/network/admin`. Zwróć uwagę na wpis dotyczący bazy danych `miner10g`.

   ```
   cd $ORACLE_HOME/network/admin
   cat tnsnames.ora
   ```

3. W jednym oknie (sesja czarna) połącz się z lokalną bazą danych jako `scott`

   ```
   sqlplus scott
   ```

4. W drugim oknie (sesja zielona) połącz się z bazą danych `miner10g` korzystając z własnego konta

   ```
   sqlplus admXX/admXX@miner
   ```

5. W sesji czarnej spróbuj utworzyć prywatne łącze bazodanowe o nazwie miner do własnego konta w bazie danych miner10g

   ```
   create database link miner
   connect to admXX identified by admXX
   using 'miner';
   ```

6. Otwórz nową sesję (niebieską) łącząc się z lokalną bazą danych jako administrator

   ```
   sqlplus / as sysdba
   ```

7. W sesji niebieskiej nadaj użytkownikowi `scott` przywilej tworzenia łączy bazodanowych oraz przywilej tworzenia synonimów

   ```
   grant create database link to scott;
   grant create synonym to scott;
   ```

8. W sesji czarnej spróbuj ponownie utworzyć łącze bazodanowe

9. W sesji czarnej odczytaj zawartość tabeli PRACOWNICY w zdalnej bazie danych miner10g poprzez utworzone łącze

```
select * from pracownicy@miner;
```

10. Utwórz synonim MPRACOWNICY dla zdalnej tabeli PRACOWNICY. Odczytaj dane poprzez utworzony synonim

```
create synonym mpracownicy for pracownicy@miner;
select * from mpracownicy;
```

Transakcje rozproszone

11. W sesji niebieskiej nadaj użytkownikowi `scott` przywilej testowania awarii i ręcznego naprawiania transakcji rozproszonych (użytkownicy w bazie `miner10g` już posiadają ten przywilej)

```
grant force any transaction to scott;
```

12. W sesji niebieskiej wyłącz automatyczne naprawianie problemów z transakcjami rozproszonymi

```
alter system disable distributed recovery;
```

13. Wykonaj transakcję rozproszoną modyfikującą dane w bazie zdalnej i lokalnej (bez awarii)

```
update pracownicy set placa_pod = placa_pod+10
where id_prac = 100;

update pracownicy@miner set placa_pod = placa_pod+10
where id_prac = 100;

commit;
```

14. Wykonaj ponownie tę samą rozproszoną transakcję symulując awarię po fazie prepare procesu 2-Phase Commit

```
update pracownicy set placa_pod = placa_pod+10
where id_prac = 100;

update pracownicy@miner set placa_pod = placa_pod+10
where id_prac = 100;

commit comment 'ORA-2PC-CRASH-TEST-4';
```

15. Spróbuj ponownie w sesji czarnej zmodyfikować dane najpierw w lokalnej a później w zdalnej bazie danych. Zaobserwuj informacje o zablokowaniu danych przez transakcję rozproszoną w stanie in-doubt

```
update pracownicy set placa_pod = placa_pod+10
where id_prac = 100;

update pracownicy@miner set placa_pod = placa_pod+10
where id_prac = 100;
```

16. W sesji niebieskiej podejrzyj informacje o transakcjach rozproszonych, które nie mogą zostać zakończone

```
select * from dba_2pc_pending;
```

17. W sesji niebieskiej włącz automatyczne naprawianie problemów z transakcjami rozproszonymi

```
alter system enable distributed recovery;
```

18. W sesji niebieskiej ponownie podejrzyj informacje o transakcjach rozproszonych, które nie mogą zostać zakończone

```
select * from dba_2pc_pending;
```

19. Jeszcze raz spróbuj ponownie w sesji czarnej zmodyfikować dane najpierw w lokalnej a później w zdalnej bazie danych. Zauważ, że tym razem operacje zakończyły się powodzeniem

```
update pracownicy set placa_pod = placa_pod+10
where id_prac = 100;

update pracownicy@miner set placa_pod = placa_pod+10
where id_prac = 100;

commit;
```

Migawki (perspektywy materializowane)

20. W sesji niebieskiej nadaj użytkownikowi `scott` przywilej tworzenia migawek

```
grant create snapshot to scott;
```

21. W sesji niebieskiej sprawdź liczbę procesów umożliwiających automatyczne odświeżanie migawek

```
show parameter job_queue_processes;
```

22. Jeśli wartość parametru `JOB_QUEUE_PROCESSES` jest równa 0, to (w sesji niebieskiej) zmień jego wartość np. na 2

```
alter system set job_queue_processes=2;
```

23. W sesji czarnej utwórz migawkę REP_ETATY będącą repliką zdalnej tabeli ETATY, odświeżaną w trybie COMPLETE bez automatycznego odświeżania.

```
create snapshot rep_etaty
refresh complete
as
select * from etaty@miner;
```

24. Odczytaj informacje o utworzonych przez siebie migawkach

```
select * from user_snapshots;
```

25. W sesji zielonej zmodyfikuj zawartość tabeli ETATY, dodając nowy etat „PORTIER”. Zatwierdź transakcję. W sesji czarnej wyświetl dane z migawki REP_ETATY.

26. W sesji czarnej odśwież „ręcznie” migawkę REP_ETATY w trybie COMPLETE. Wyświetl dane z migawki.

```
exec dbms_snapshot.refresh('SCOTT.REP_ETATY', 'C')
select * from rep_etaty;
```

27. W sesji zielonej utwórz dziennik migawki na tabeli PRACOWNICY. Dziennik ten pozwoli na przyrostowe (FAST) odświeżanie migawek replikujących dane z tej tabeli.

```
create snapshot log on pracownicy;
```

28. W sesji czarnej utwórz migawkę REP_PLACE replikującą identyfikatory, nazwiska, płace podstawowe i dodatkowe ze zdalnej tabeli PRACOWNICY. Migawka powinna być odświeżana w trybie FAST automatycznie co 6 minut.

```
create snapshot rep_place
refresh fast
next sysdate + 1/240
as
select id_prac, nazwisko, placa_pod, placa_dod
from pracownicy@miner;
```

29. W sesji czarnej wyświetl dane z migawki REP_PLACE

```
select * from rep_place;
```

30. W sesji zielonej zmodyfikuj płacę któremuś z pracowników. Zatwierdź transakcję!

31. W sesji czarnej wyświetl dane z migawki REP_PLACE. Następnie odśwież "ręcznie" migawkę REP_PLACE w trybie FAST. Ponownie wyświetl dane z migawki REP_PLACE

```
select * from rep_place;
exec dbms_snapshot.refresh('SCOTT.REP_PLACE', 'F')
select * from rep_place;
```

32. W sesji zielonej dodaj nowego pracownika. Zatwierdź transakcję!

33. W sesji czarnej wyświetlaj co jakiś czas zawartość migawki REP_PLACE, czekając na automatyczne odświeżenie migawki.

34. W sesji czarnej utwórz grupę odświeżania zawierającą migawki REP_ETATY i REP_PLACE (pierwsze odświeżenie: natychmiast po utworzeniu, okres odświeżania: co 6 minut, zmiana grupy odświeżania dla migawek już automatycznie odświeżanych).

```
exec DBMS_REFRESH.MAKE (name => 'scott.rg_kadry', -
  list => 'scott.rep_etaty, scott.rep_place', -
  next_date => sysdate, -
  interval => 'sysdate + 1/240', -
  lax => true)
```

35. W sesji zielonej usuń nowego pracownika dodanego w punkcie 32. Zatwierdź transakcję!

36. W sesji czarnej odczytaj dane z migawki REP_PLACE. Odśwież „ręcznie” grupę odświeżania. Ponownie odczytaj dane z migawki REP_PLACE

```
select * from rep_place;
exec DBMS_REFRESH.REFRESH ('scott.rg_kadry')
select * from rep_place;
```

37. W sesji zielonej usuń z tabeli ETATY etat „PORTIER”. Zatwierdź transakcję.

38. W sesji czarnej wyświetlaj co jakiś czas zawartość migawki REP_ETATY, czekając na automatyczne odświeżenie migawek z grupy odświeżania.

39. W sesji czarnej usuń grupę odświeżania, a następnie obie utworzone wcześniej migawki

```
exec DBMS_REFRESH.DESTROY ('scott.rg_kadry')
drop snapshot rep_place;
drop snapshot rep_etaty;
```

40. W sesji zielonej usuń dziennik migawki na tabeli PRACOWNICY

```
drop snapshot log on pracownicy;
```